python-zero-to-projects

Chapter 3: Operators

Complete Study Notes

Prerequisites: Chapter 1 (Introduction) and Chapter 2 (Variables and Data Types).

Quick Review: Variables from Chapter 2

In Chapter 2, you learned to store values in variables using four core data types. Every operator example in this chapter uses these same variables so the transition feels natural.

```
# Variables declared in Chapter 2 style -- we will use these throughout Chapter
3
a = 10          # int
b = 3           # int
x = 5.5         # float
name = "Alice"  # str
flag = True     # bool

print(type(a))    # <class 'int'>
print(type(x))    # <class 'float'>
print(type(name)) # <class 'str'>
print(type(flag)) # <class 'bool'>
```

Section 1: Arithmetic Operators

Arithmetic operators perform mathematical calculations. Python supports seven arithmetic operators.

1.1 Addition (+)

```
a = 10
b = 3
result = a + b
print(result)    # 13
```

1.2 Subtraction (-)

```
result = a - b
```

```
print(result)    # 7
```

1.3 Multiplication (*)

```
result = a * b
print(result)    # 30
```

1.4 True Division (/)

The / operator ALWAYS returns a float, even when both operands are integers and the result is a whole number. This is different from many older languages.

```
result = a / b
print(result)    # 3.3333333333333335  (float)

result2 = 10 / 2
print(result2)   # 5.0  (still a float, NOT 5)
```

> *Note: / always returns float. Use // if you need an integer result.*

1.5 Floor Division (//)

Floor division divides and rounds DOWN to the nearest integer. If both operands are int, the result is int. If either operand is float, the result is float but still rounded down.

```
result = a // b
print(result)    # 3  (int, not 3.333...)

result2 = 10.0 // 3
print(result2)   # 3.0  (float, but still rounded down)

result3 = -7 // 2
print(result3)   # -4  (rounds DOWN toward negative infinity, not toward zero)
```

> *Note: Floor division always rounds toward NEGATIVE infinity. So -7 // 2 gives -4, not -3.*

1.6 Modulo (%)

The % operator returns the REMAINDER after division. This is very useful for checking if a number is even or odd, or for cycling through a range.

```
result = a % b
print(result)    # 1  (10 divided by 3 is 3 remainder 1)

print(10 % 2)    # 0  (10 is divisible by 2 with no remainder)
print(11 % 2)    # 1  (11 is odd)
print(7 % 3)     # 1
```

1.7 Exponentiation (**)

The ** operator raises the left operand to the power of the right operand.

```
result = a ** b
print(result)    # 1000  (10 to the power of 3)

print(2 ** 8)    # 256
print(9 ** 0.5)  # 3.0  (square root of 9)
print(2 ** -1)   # 0.5  (negative exponent = reciprocal)
```

Section 2: Comparison Operators

Comparison operators compare two values and always return a boolean (True or False). They are the foundation of decision-making in code.

2.1 Overview Table

```
a = 10
b = 3

print(a == b)   # False  -- equal to
print(a != b)   # True   -- not equal to
print(a >  b)   # True   -- greater than
print(a <  b)   # False  -- less than
print(a >= b)   # True   -- greater than or equal to
print(a <= b)   # False  -- less than or equal to
```

2.2 Comparing Floats

```
x = 5.5
print(x > 5)     # True
print(x == 5.5)  # True
print(x != 5)    # True
```

2.3 Comparing Strings (lexicographic order)

Strings are compared character by character using their Unicode code points (essentially alphabetical order). Capital letters come before lowercase.

```
name = "Alice"
print(name == "Alice")   # True
print(name == "alice")   # False  (case-sensitive)
print(name != "Bob")     # True
print("apple" < "banana")  # True  (a comes before b)
```

```
print("Z" < "a")          # True  (uppercase Z has a lower code point than 'a')
```

2.4 Comparing Booleans

```
flag = True
print(flag == True)    # True
print(flag == 1)       # True  (True is treated as 1 internally)
print(flag == False)   # False
```

2.5 Chaining Comparisons

Python allows chaining comparisons in a mathematically natural way:

```
score = 75
print(0 <= score <= 100)   # True  (score is between 0 and 100, inclusive)
print(10 < score < 80)     # True
```

Section 3: Logical Operators

Logical operators combine boolean expressions. Python uses the English words and, or, and not (unlike many other languages that use &&, ||, !).

3.1 and

Returns True only when BOTH sides are True.

```
# Truth table for 'and'
# True  and True  -> True
# True  and False -> False
# False and True  -> False
# False and False -> False

a = 10
b = 3
print(a > 5 and b < 10)   # True  (both True)
print(a > 5 and b > 10)   # False (second is False)
print(a < 5 and b < 10)   # False (first is False)
```

3.2 or

Returns True when AT LEAST ONE side is True.

```
# Truth table for 'or'
# True  or True  -> True
# True  or False -> True
```

```
# False or True  -> True
# False or False -> False

print(a > 5 or b > 10)   # True  (first is True)
print(a < 5 or b < 10)   # True  (second is True)
print(a < 5 or b > 10)   # False (both False)
```

3.3 not

Reverses (negates) a boolean value.

```
# Truth table for 'not'
# not True  -> False
# not False -> True

flag = True
print(not flag)          # False
print(not (a < 5))       # True  (a < 5 is False, so not False is True)
print(not (a > 5))       # False
```

3.4 Combining Logical Operators

```
a = 10
b = 3
x = 5.5

# Complex condition
result = (a > 5) and (b < 10) and (x != 0)
print(result)   # True

# Short-circuit evaluation:
# Python stops evaluating 'and' as soon as it finds a False.
# Python stops evaluating 'or' as soon as it finds a True.
print(False and (10 / 0))  # False -- division never happens!
print(True or (10 / 0))    # True  -- division never happens!
```

Section 4: Assignment Operators

Assignment operators store values in variables. Compound assignment operators are shorthand for updating a variable.

```
a = 10          # Basic assignment

# Augmented (compound) assignment operators:
a += 5          # Same as: a = a + 5       -> a is now 15
```

```
a -= 3          # Same as: a = a - 3       -> a is now 12
a *= 2          # Same as: a = a * 2       -> a is now 24
a /= 4          # Same as: a = a / 4       -> a is now 6.0  (float!)
a //= 2         # Same as: a = a // 2      -> a is now 3.0
a %= 2          # Same as: a = a % 2       -> a is now 1.0
a **= 3         # Same as: a = a ** 3      -> a is now 1.0

# Starting fresh with integer examples:
score = 0
score += 10     # Player earns 10 points -> score is 10
score += 5      # Player earns 5 more    -> score is 15
score -= 2      # Penalty of 2           -> score is 13
print(score)    # 13
```

> *Note: /= always produces a float result even if the original variable was an int. Use //= if you need to stay with integers.*

Section 5: Bitwise Operators

Bitwise operators work directly on the binary (base-2) representation of integers. Each bit is processed independently.

5.1 Quick Binary Primer

In binary, every integer is stored as a sequence of 0s and 1s. For example:

```
# Decimal  Binary (8-bit)   Python bin() function
#   60     00111100          bin(60)  -> '0b111100'
#   15     00001111          bin(15)  -> '0b1111'

print(bin(60))  # 0b111100
print(bin(15))  # 0b1111
```

5.2 Bitwise AND (&)

Compares each pair of bits. Result bit is 1 only if BOTH bits are 1.

```
a = 60    # 00111100
b = 15    # 00001111
result = a & b
print(result)    # 12  (00001100)
print(bin(result))  # 0b1100
```

5.3 Bitwise OR (|)

Result bit is 1 if AT LEAST ONE of the bits is 1.

```
result = a | b
print(result)    # 63  (00111111)
print(bin(result))  # 0b111111
```

5.4 Bitwise XOR (^)

Result bit is 1 if the bits are DIFFERENT (one 0, one 1).

```
result = a ^ b
print(result)    # 51  (00110011)
print(bin(result))  # 0b110011
```

5.5 Bitwise NOT (~)

Flips all bits. For a positive integer n, ~n equals -(n+1) due to how Python represents negative numbers (two's complement).

```
result = ~a
print(result)    # -61  (flips all bits of 60)
```

5.6 Left Shift (<<)

Shifts all bits to the left by n positions. Each left shift by 1 is equivalent to multiplying by 2.

```
result = a << 2
print(result)    # 240  (00111100 becomes 11110000)
# 60 * (2**2) = 60 * 4 = 240
```

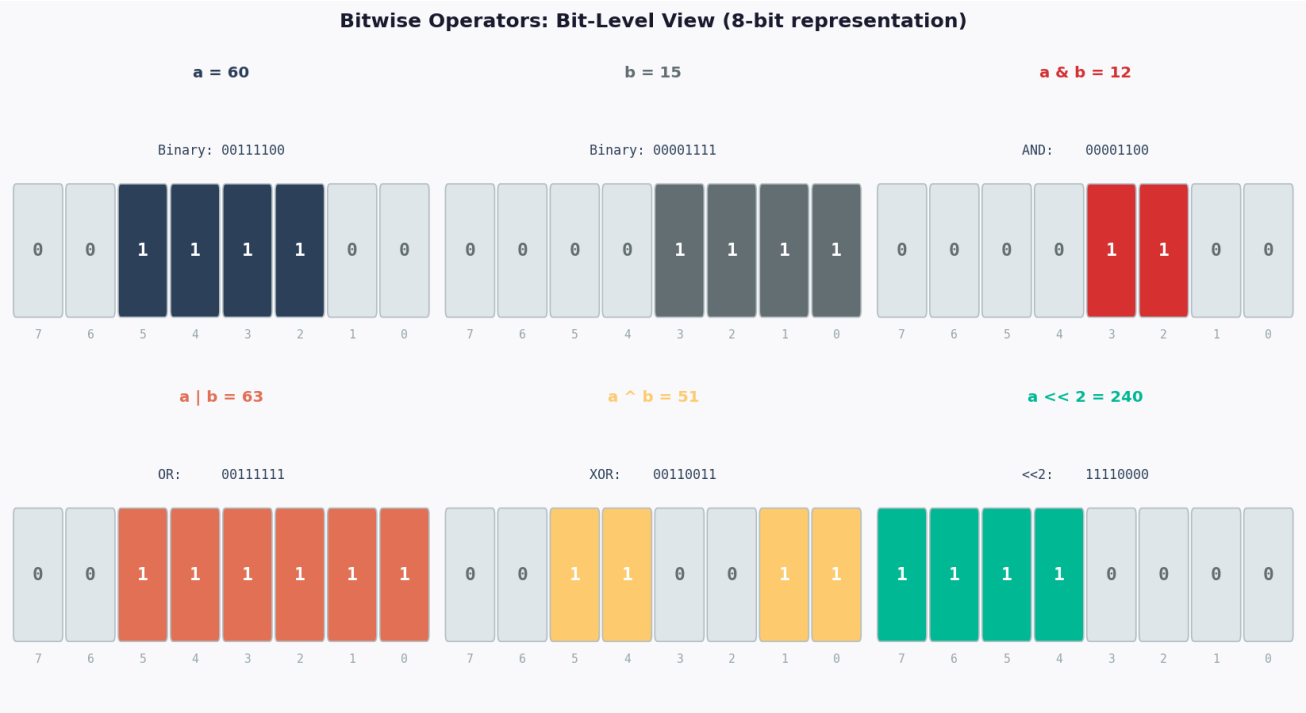
5.7 Right Shift (>>)

Shifts all bits to the right by n positions. Each right shift by 1 is equivalent to integer division by 2.

```
result = a >> 2
print(result)    # 15  (00111100 becomes 00001111)
# 60 // (2**2) = 60 // 4 = 15
```

5.8 Bitwise Diagram

The diagram below shows the bit patterns for these operations using a = 60 and b = 15:



Section 6: Operator Precedence

When an expression contains multiple operators, Python uses a fixed order of precedence (similar to PEMDAS in mathematics) to decide which operation to perform first.

6.1 Precedence Table (Highest to Lowest)

```
# Precedence (highest first):
#   1. Parentheses                    ()
#   2. Exponentiation                 **
#   3. Unary positive / negative      +x, -x, ~x
#   4. Multiplication, Division,
#      Floor Division, Modulo         *  /  //  %
#   5. Addition, Subtraction          +  -
#   6. Bitwise Shifts                 <<  >>
#   7. Bitwise AND                    &
#   8. Bitwise XOR                    ^
#   9. Bitwise OR                     |
#  10. Comparisons                    ==  !=  >  <  >=  <=
#  11. Logical NOT                    not
#  12. Logical AND                    and
#  13. Logical OR                     or
#  14. Assignment                     =  +=  -=  etc.
#
# Operators at the same level are evaluated left-to-right (left associative),
# EXCEPT ** which is right-to-left (right associative).
```

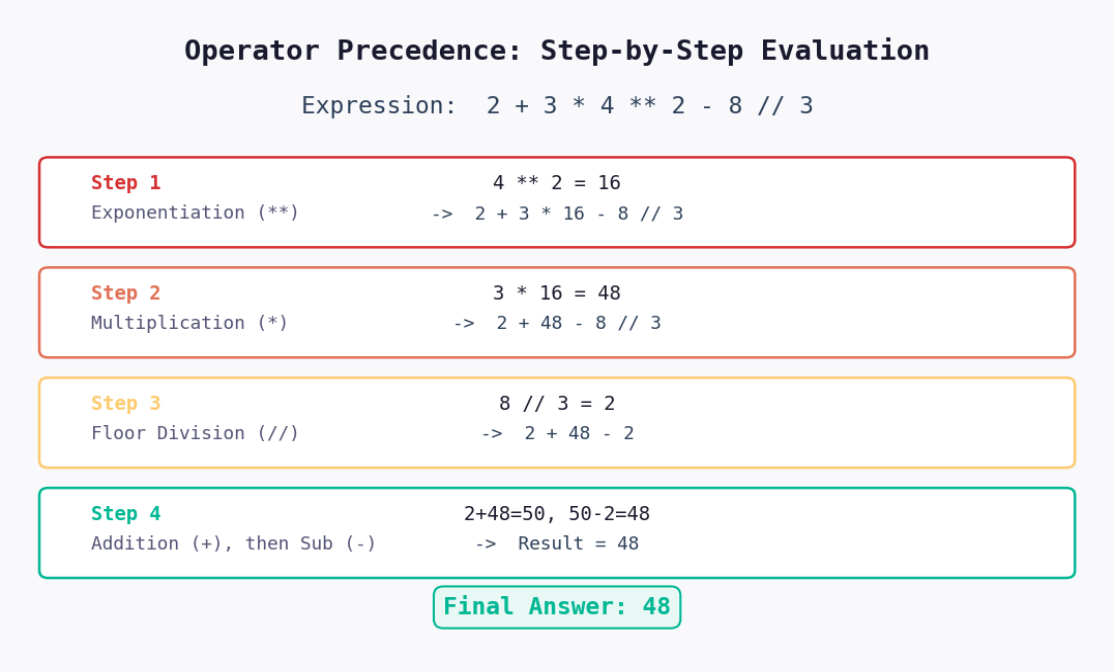
6.2 Step-by-Step Example

Let's evaluate: 2 + 3 * 4 ** 2 - 8 // 3

```
expr = 2 + 3 * 4 ** 2 - 8 // 3

# Step 1: Exponentiation first       4 ** 2 = 16
#         -> 2 + 3 * 16 - 8 // 3
# Step 2: Multiplication             3 * 16 = 48
#         -> 2 + 48 - 8 // 3
# Step 3: Floor Division             8 // 3 = 2
#         -> 2 + 48 - 2
# Step 4: Addition and Subtraction (left to right)
#         2 + 48 = 50  -> 50 - 2 = 48
# Result: 48

print(expr)   # 48
```



6.3 Using Parentheses to Override Precedence

```
print(2 + 3 * 4)       # 14   (multiplication first)
print((2 + 3) * 4)     # 20   (parentheses override)

print(10 - 4 - 2)      # 4    (left to right)
print(10 - (4 - 2))    # 8    (parentheses change order)

# Rule: when in doubt, use parentheses to make your intent clear.
```

Section 7: Common Beginner Mistakes

7.1 Using = Instead of == in Comparisons

The single = is assignment; == is comparison. This is one of the most common beginner errors.

```
# WRONG -- this is assignment, not comparison
# if x = 5:     <- SyntaxError in Python (caught by the interpreter)

# CORRECT -- use == for comparison
x = 5
if x == 5:
    print("x is 5")

# Exception: Python will raise a SyntaxError if you use = inside if/while,
# so this mistake is usually caught immediately.
```

7.2 Division Surprises: / vs //

```
# Expecting an integer but getting a float:
result = 10 / 2
print(result)          # 5.0  (float, not 5)
print(type(result))    # <class 'float'>

# Fix: use // if you need an integer
result = 10 // 2
print(result)          # 5    (int)
```

7.3 Operator Precedence Confusion

```
# Thinking addition happens before multiplication:
print(2 + 3 * 4)       # Beginners may expect 20, but answer is 14

# Fix: use parentheses to enforce intended order
print((2 + 3) * 4)     # 20 -- now it is clear
```

7.4 Floor Division of Negative Numbers

```
# Expecting -3 but getting -4:
print(-7 // 2)   # -4  (rounds toward negative infinity, not toward zero)

# If you need truncation toward zero, use int():
import math
print(int(-7 / 2))   # -3  (truncates toward zero)
```

7.5 Mixing Bitwise and Logical Operators

```
# WRONG intent -- & has higher precedence than ==, causing unexpected results
# if a & b == 0:    <- Python reads this as: a & (b == 0)

# CORRECT -- use parentheses
a = 60
b = 15
if (a & b) == 0:
    print("No bits in common")
else:
    print("Shared bits exist")  # This prints: 60 & 15 = 12, not 0

# Use 'and' / 'or' for logical conditions, & / | only for bit manipulation.
```

7.6 Modulo with Negative Numbers

```
# Python's modulo result always has the same sign as the divisor (b)
print(-10 % 3)    # 2  (not -1 as you might expect)
print(10 % -3)    # -2 (not 1)

# This is consistent with floor division:
# -10 = (-4) * 3 + 2  ->  -10 // 3 is -4, -10 % 3 is 2
```

Section 8: Summary and Recap

Here is a quick reference of everything covered in Chapter 3:

```
# ARITHMETIC (return numbers)
# +    addition         a + b
# -    subtraction      a - b
# *    multiplication   a * b
# /    true division    a / b   (always float)
# //   floor division   a // b  (rounds down)
# %    modulo           a % b   (remainder)
# **   exponentiation   a ** b

# COMPARISON (return bool)
# ==   equal to
# !=   not equal to
# >    greater than
# <    less than
# >=   greater than or equal to
# <=   less than or equal to

# LOGICAL (combine booleans)
```

```
# and  True if both sides True
# or   True if at least one side True
# not  reverses True/False

# ASSIGNMENT (store / update variables)
# =   +=   -=   *=   /=   //=   %=   **=

# BITWISE (bit-level operations on integers)
# &    AND    |    OR    ^    XOR
# ~    NOT    <<   left shift    >>   right shift

# PRECEDENCE ORDER (simplified):
# ()  ->  **  ->  unary  ->  * / // %  ->  + -
# ->  shifts  ->  &  ->  ^  ->  |
# ->  comparisons  ->  not  ->  and  ->  or
```

Preview: What Comes Next in Chapter 4 (Strings)

In Chapter 4, you will discover that some of these same operators work with strings in new and powerful ways:

- The + operator becomes string CONCATENATION: "Hello" + " " + "World" gives "Hello World"
- The * operator becomes string REPETITION: "ha" * 3 gives "hahaha"
- The == and != operators compare strings: "apple" == "apple" gives True
- The > and < operators sort strings alphabetically: "apple" < "banana" gives True
- The in operator (new in Chapter 4) checks substrings: "lo" in "Hello" gives True

The operators you have learned here are not limited to numbers -- they form the core language of Python that applies to nearly every data type you will encounter.